

Abstract Data Types

Programming Languages

Millersville University

Outline

- Abstraction
- Abstract Data Types
- Data Encapsulation & Hiding
- Constructors and Destructors
- Accessors and Mutators
- Encapsulations
- Naming Encapsulations

The image features a solid orange background. A large white circle is centered on the page. To the left of the white circle, there is a yellow dashed line that forms a partial arc. At the bottom right of the white circle, there is a small blue solid circle.

Abstraction

Abstraction

- An *abstraction* is a view or representation of an entity that includes only the most significant attributes
- It is fundamental in programming (and CS)
- Nearly all programming languages support process abstraction with *subprograms*
- Nearly all programming languages designed since 1980 support *data abstraction*

Abstract Data Types

Abstract Data Type (ADT)

User-defined data type that satisfies two conditions:

1. Representation is *hidden* from clients of the type. Only operations possible are those provided in the definition.
2. Declarations of the state and supported operations are contained within a *single syntactic unit*.

ADT Advantages

1. The representation is hidden from clients of the type. Only operations possible are those provided in the definition.

- Reliable – by hiding the data representation, user code cannot directly access objects of the type or depend on the representation
- Modular – can focus on smaller range of code and variables
- Readable – name conflicts are less likely in an “owned” code model

ADT Advantages

2. The declarations of the state and supported operations are contained within a single syntactic unit.

- Provides method of program organization
- Aids modifiability (related info is grouped)
- Separate compilation
 - For massive projects, this is crucial

Data Encapsulation

- Condition #1 is a form of *Information Hiding*
each unit hides a design decision
- Condition #2 is a form of *Encapsulation*
all members are in a single unit

```
class Player
    hp      : int
    name     : string
    position : vec3
    fn move (d : vec3)
```


Data/Information Hiding

- Should (must?) be able to hide implementation details (state and operations) from clients
- Introduce visibility modifiers to the language

public

member is accessible to clients

private

member is only accessible within the class

protected

private + visible to derived classes

Object Init: Constructors

- Special functions that allow the user to *initialize* the data members of objects
- Can include parameters to provide initial state
- Implicitly called upon creation of an object
- Can also be explicitly called!
- Name is same as class name (C++, Java)

Object Destruction: Destructors

- Special function that cleans up an instance when it's destroyed
 - Release resources acquired during object's lifetime (like ?)
- Implicitly called when the instance's lifetime ends

- Can also be explicitly called!

```
void* buffer = malloc(sizeof(T));  
T* t1 = new(buffer) T();  
...  
t1->~T();  
free(buffer);
```

- Only exist in languages that require manual resource management

END OF EXAM CONTENT

- END OF EXAM CONTENT

Object Access: Accessor Methods

- Methods that *read* state (never modify)
- Often named as **noun()** or **getNoun()**
- Immutability: state must not change when invoked (violation of contract)
- Some languages allow us to force the contract of zero instance modification (**const** in C++)

Object Mutation: Mutator Methods

- Methods that *modify* state in some way
- Often named as **verb()** or **setNoun()**
- Often will require parameters
- Mutable: one or more instance variables will be changed. The “state” of the program evolves as an instance of a class changes.

Object Properties

- Enable a programmer to specify accessors and mutators more concisely than the typical three parts:

- | | |
|--------------------------|-----------------------|
| • private data | name |
| • public accessor method | getName |
| • public mutator method | setName(name: string) |

- Often used to condense validation code to a single area

- Can create “artificial” entities that correspond to data-backed members

```
player.Health = 800;
```

Object Properties in C#

```
class Time { ...  
    public double Hours {  
        get { return seconds / 3600; }  
  
        set {  
            if (value < 0 || value > 24)  
                seconds = value * 3600;  
            else  
                throw new ArgumentException(...);  
        }  
    }  
    private int seconds;  
}
```


Encapsulations

- Large programs have two special needs
 1. Some means of organization, other than giant collection of subpgms. and ADTs
 2. Some means of partial compilation (programming units that are smaller than the entire program)
- Solution:
 - Group a collection of logically related subprograms and compile them as a unit
 - This group is called an *encapsulation*

Nested Subprograms

- Nest subprograms within the larger subprograms that use them
- Nested subprograms are supported in Python, JavaScript, and Ruby

Encapsulation in C

- Files containing one or more subprograms can be independently compiled
- Interface is placed in a *header file*
- Implementation is placed in *... implementation file*
- Numerous problems, like multiple definitions, repetition
- Include directive used to include headers in other source files

Encapsulation in C++

- Can define header and source files, similar to those of C
- Or, classes can be used for encapsulation
 - Class is used as the interface (prototypes)
 - Member definitions are defined in a separate file
- C++20 supports modules

C# Assemblies

- *Assembly*: collection of files that appears to application programs to be a single dynamic link library or executable
- Defines a module that can be separately compiled
- Contains code (CIL), class metadata, list of referenced assemblies, version data, etc.

Naming Encapsulations

- Large programs define many *global* names with possibility to clash
(e.g. two entities that use the name **Vector**)
- Need way to divide them into logical groupings
- A *naming encapsulation* is used to create a new scope for names
 - What encapsulations are provided by C++, Java, OCaml?

Naming Encapsulations

- Languages provide different encapsulations
 - C++ – *namespaces*
 - Everything in the std namespace is owned by the C++ Standard
 - Everything in the glm namespace is maintained by the glm library
 - Java – *packages*
 - java.* is part of the base Java standard library
 - javax.* is part of the Java eXtensions to the standard library
 - C# – *namespaces*
 - Like C++ namespaces (System is a built-in namespace)
 - Ruby – *modules*

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz
modified by Gary Zoppetti

Sources include Robert Sebesta's [Concepts of Programming Languages](#)